

Come e perché funzionano le cose: dagli agenti al machine learning

Corso di Sistemi Intelligenti, Cristina Baroglio — Università di Torino

Moduli 1–8

Indice

1	Agenti Intelligenti	2
1.1	Cosa vuol dire “intelligenza artificiale”	2
1.1.1	Il Test di Turing e i suoi limiti	2
1.1.2	Due modi di intendere l’obiettivo dell’IA	3
1.2	L’astrazione centrale: agente e ambiente	3
1.2.1	Cosa vuol dire essere razionali	3
1.2.2	PEAS: il primo passo di ogni progettazione	4
1.2.3	Le proprietà dell’ambiente: perché contano	4
1.3	Le cinque tipologie di agente	5
1.4	Dal come al cosa: il paradigma dichiarativo	6
2	Ricerca nello spazio degli stati	7
2.1	L’idea di fondo	7
2.2	Ricerca non informata: un confronto ragionato	7
2.3	Ricerca informata: perché la conoscenza aiuta	8
2.3.1	Da dove vengono le buone euristiche	9
3	Ricerca con avversario: i giochi	10
3.1	Perché i giochi interessano l’IA	10
3.2	Minimax	10
3.3	Potatura alfa-beta: rendere Minimax praticabile	11
3.4	Quando non si può guardare fino in fondo	11
4	Constraint Satisfaction Problems (CSP)	12
4.1	Perché aprire la scatola nera	12
4.2	Il trucco decisivo: la commutatività	12
4.3	Euristiche: dove rischiare e come minimizzare il rischio	12
4.4	Propagazione dei vincoli: guardare avanti prima di sbagliare	13
4.5	Oltre il backtracking cronologico	13
5	Logica proposizionale e rappresentazione della conoscenza	14
5.1	Perché serve un linguaggio formale	14

5.2	I concetti cardine: modello, conseguenza, inferenza	14
5.3	Sintassi e semantica proposizionale	14
5.4	Come verificare se KB implica qualcosa	15
5.5	Clausole di Horn: quando l'inferenza diventa veloce	15
6	Logica del primo ordine (FOL)	16
6.1	Il limite della proposizionale	16
6.2	L'errore più comune: abbinare male i quantificatori	16
6.3	L'ordine dei quantificatori conta	17
6.4	Dall'inferenza al lifting	17
6.5	Skolemizzazione: l'errore da non fare mai	17
7	Ontologie e Pianificazione automatica	19
7.1	Dalla tassonomia all'ontologia	19
7.2	Perché serve un formalismo diverso per pianificare	19
7.3	Il frame problem	20
7.4	L'anomalia di Sussman: quando i sottoobiettivi litigano	20
8	Machine Learning: classificazione, alberi di decisione, reti neurali	21
8.1	Le classi non esistono in natura	21
8.2	Alberi di decisione	21
8.3	Il perceptron: un iperpiano che impara	22
8.4	MLP e backpropagation	22

1 Agenti Intelligenti

1.1 Cosa vuol dire “intelligenza artificiale”

Il punto di partenza del corso è disarmante: se chiedi a due sistemi di IA generativa (per dire, ChatGPT e DeepSeek) chi fosse un personaggio storico che semplicemente non esiste, entrambi ti rispondono con sicurezza, in modo fluente, e inventano due biografie completamente diverse e completamente false. Questo è il punto pedagogico da cui parte tutto il corso: un sistema può produrre output linguisticamente perfetti senza che questo implichi che *capisca* qualcosa di ciò che dice. Tenete a mente questa distinzione tra *comportarsi come se* e *capire davvero*, perché ci accompagnerà per tutto il modulo.

Cos'è allora l'intelligenza artificiale? Semplicemente: una forma di intelligenza *non naturale*, cioè costruita dall'uomo con mezzi informatici invece che sviluppata biologicamente. Ma qui casca subito un asino concettuale che le slide vogliono farvi superare: **automazione non è sinonimo di intelligenza**. Una calcolatrice esegue automaticamente una sequenza di istruzioni, ma nessuno la definirebbe intelligente, perché non si adatta a nulla di imprevisto: fa sempre e solo quello per cui è stata programmata. Per parlare di intelligenza serve qualcosa in più: la capacità di **adattarsi**, di scegliere l'azione più opportuna anche in situazioni che il programmatore non aveva previsto esplicitamente, ed eventualmente di **imparare** dall'esperienza. È esattamente questo salto concettuale — da “programma che esegue” ad “agente che delibera” — il filo conduttore di tutto il modulo.

Storicamente, la disciplina nasce ufficialmente nel 1956 al *Dartmouth Summer Research Project*, dove John McCarthy conia il termine *Artificial Intelligence*. Prima di allora, però, c'era già stato un episodio fondamentale: nel 1950 Alan Turing pubblica “Computing Machinery and Intelligence”, in cui propone di sostituire la domanda mal posta “una macchina può pensare?” con una domanda operativa, il celebre **Test di Turing**.

1.1.1 Il Test di Turing e i suoi limiti

L'idea del test è semplice da descrivere: un interrogatore umano scambia messaggi scritti con due interlocutori nascosti, uno umano e uno macchina, senza sapere quale sia quale. Se l'interrogatore non riesce a distinguerli in modo affidabile, la macchina ha superato il test.

Il punto delicato, però, è che il test valuta *solo il comportamento osservabile*: guarda input e output, non cosa succede “dentro”. Le slide portano un esempio molto efficace per farvi toccare con mano il problema: due persone, Valeria e Rossella, rispondono entrambe correttamente “Piemonte” alla domanda “dove si trova Torino?”. Valeria lo sa perché ha studiato geografia e ha ragionato; Rossella ha semplicemente tirato a indovinare ed è stata fortunata. Dall'esterno, le due risposte sono identiche e indistinguibili — eppure solo una delle due implica vera comprensione. Ecco perché il Test di Turing è considerato necessario ma probabilmente non sufficiente per parlare di intelligenza in senso forte: **stesso output non implica stessa comprensione**.

John Searle, nel 1980, rincarà la dose con l'esperimento mentale della **Stanza Cinese**: immaginate una persona che non conosce il cinese, chiusa in una stanza, che riceve frasi in ideogrammi e — seguendo meccanicamente un manuale di regole puramente sintattiche (che dice “se vedi questa sequenza di simboli, scrivi quest'altra”) — produce risposte in cinese perfettamente sensate. Chi sta fuori crede di conversare con qualcuno che capisce il cinese. Ma la persona dentro la stanza sta solo manipolando simboli in base alla loro forma, senza avere accesso al loro significato. La tesi di Searle è che questo è esattamente ciò che fa un computer: manipola simboli sintatticamente, mai semanticamente — eseguire un programma non è mai, di per sé, condizione sufficiente per avere una vera comprensione (intenzionalità).

1.1.2 Due modi di intendere l'obiettivo dell'IA

Da questa discussione filosofica nascono due modi radicalmente diversi di porsi l'obiettivo della disciplina:

- la **Strong AI** vuole davvero riprodurre il pensiero umano: il criterio di successo è che la macchina pensi/capisca come un umano, ed è quindi legata alle scienze cognitive;
- la **Weak AI** è invece *task-oriented*: le basta che il sistema produca la soluzione corretta, indipendentemente dal meccanismo interno che la genera. Il criterio di successo è il *comportamento razionale*, non necessariamente umano.

Il corso adotta esplicitamente questo secondo approccio (in linea con Russell & Norvig), perché è molto più operativo: per progettare un agente non serve risolvere il problema filosofico della coscienza, basta specificare un comportamento che “faccia la cosa giusta”.

1.2 L'astrazione centrale: agente e ambiente

Tutta l'architettura concettuale del modulo ruota attorno a un'unica immagine: un **agente** è un qualsiasi sistema che *percepisce* il proprio ambiente tramite sensori e *agisce* su di esso tramite attuatori. Non ha senso parlare di un agente isolato dal suo ambiente: sono un binomio inscindibile. Tra percezione e azione c'è sempre una funzione deliberativa (spesso disegnata come un punto interrogativo nei diagrammi) che decide cosa fare sulla base di ciò che è stato percepito — ed è proprio il modo in cui questa funzione viene implementata a distinguere i diversi tipi di agente che vedremo tra poco.

Un concetto tecnico ma importante è quello di **sequenza percettiva**: la storia completa di tutto ciò che l'agente ha percepito fino a un certo istante. In linea di principio, il modo più generale (anche se costosissimo) per definire il comportamento di un agente sarebbe una tabella che associa ad ogni possibile sequenza percettiva un'azione. Perché questo concetto è importante? Perché fa emergere subito una domanda cruciale: se due aspirapolvere robotici si comportano in modo diverso nello stesso identico ambiente, quale dei due è “migliore”? Non possiamo rispondere senza un criterio oggettivo — ed è qui che entra in gioco la razionalità.

1.2.1 Cosa vuol dire essere razionali

Informalmente, un agente razionale è quello che “fa la cosa giusta”. Per renderlo preciso serve una **misura di prestazione** (performance measure): un criterio che valuta la bontà degli stati attraversati dall'ambiente, in funzione dell'effetto desiderato — non della singola azione presa isolatamente.

La definizione formale che dovete saper riportare è questa:

Un agente razionale dovrebbe scegliere, per ogni sequenza percettiva, l'azione che **massimizza la misura di prestazione attesa**, data la sequenza percettiva stessa e la conoscenza dell'ambiente posseduta dall'agente.

La parola chiave è *attesa*. Questo è il punto su cui l'esame ama insistere: **razionalità non vuol dire successo, né tantomeno onniscienza**. L'esempio classico è quello della torta di compleanno: volete comprare la torta migliore e meno costosa per il compleanno di vostro figlio, conoscete le offerte dei pasticceri della zona, i gusti del bambino, e scegliete di conseguenza — un comportamento perfettamente razionale. Ma due cose potevano sfuggirvi: che la nonna, a vostra insaputa, porta una torta identica, oppure che esiste una nuova pasticceria con un'offerta migliore che non conosceste. In entrambi i casi il risultato finale non è ottimale in assoluto, ma il vostro comportamento resta razionale, perché avete fatto la scelta migliore possibile *dato ciò*

che sapevate. Un agente onnisciente, che sapesse tutto in anticipo, avrebbe ottenuto il risultato migliore in assoluto — ma la razionalità non richiede onniscienza, richiede solo di ottimizzare rispetto alle informazioni disponibili.

C'è però una precisazione interessante legata all'apprendimento: se l'anno successivo, sapendo ormai che la nonna porta sempre la torta perfetta, comprate comunque una torta senza verificare prima, il vostro comportamento *non è più* razionale — perché quella conoscenza fa ormai parte della vostra esperienza pregressa. Questo mostra che un agente razionale diventa via via più efficace quanto più impara dall'esperienza passata.

Un'altra sottigliezza: la percezione non è un atto passivo. È razionale attraversare la strada senza prima guardare a destra e sinistra? Ovviamente no — un agente razionale, prima di agire, valuta se ha raccolto abbastanza informazioni oppure se conviene compiere ulteriori atti percettivi. La razionalità dipende dalla sequenza percettiva, ma può anche *orientarla*: un buon agente sceglie attivamente di percepire di più prima di decidere.

1.2.2 PEAS: il primo passo di ogni progettazione

Quando dovete progettare un agente (anche in un esercizio d'esame), il primo passo sistematico è sempre specificare il **PEAS**:

- **P**erformance measure — il criterio con cui giudicare il successo;
- **E**nvironment — l'ambiente in cui l'agente opera;
- **A**ctuators — cosa può fare l'agente;
- **S**ensors — cosa può percepire.

Specificare il PEAS obbliga a chiarire, prima di tutto il resto, cosa l'agente deve davvero ottenere e con quali mezzi — è un esercizio che vi conviene saper fare a mente anche su esempi non visti a lezione.

1.2.3 Le proprietà dell'ambiente: perché contano

Non tutti gli ambienti sono ugualmente difficili da affrontare, e le tecniche di progettazione necessarie dipendono proprio da *quanto* è complesso l'ambiente. Le slide identificano sette assi indipendenti lungo cui classificare un ambiente:

Asse	Valore facile	Valore difficile
Osservabilità	completa (i sensori danno accesso a tutto il rilevante)	parziale
Determinismo	deterministico	stocastico (strategico se dipende solo da altri agenti)
Episodicità	episodico (percezione-azione indipendenti)	sequenziale
Dinamicità	statico (il mondo non cambia mentre l'agente decide)	dinamico
Discretezza	discreto	continuo
N. agenti	singolo agente	multiagente

Una connessione sottile ma importante: spesso un ambiente *appare* stocastico proprio perché è parzialmente osservabile. Esempio: una batteria si scarica in modo continuo, ma l'agente percepisce solo tre livelli discreti (basso/medio/alto); la stessa azione “fai un passo”, eseguita quando il livello percepito è “medio”, a volte porta a “basso” e a volte lascia il livello a “medio” — non perché il mondo sia davvero casuale, ma perché l'agente non vede il valore

esatto e continuo di carica residua. Il caso più difficile in assoluto (e anche il più realistico, pensate a un'automobile a guida autonoma) è un ambiente parzialmente osservabile, stocastico, sequenziale, dinamico, continuo e multiagente.

1.3 Le cinque tipologie di agente

Questa è probabilmente la tassonomia più citata in sede d'esame, perché mostra un percorso di crescente sofisticazione — ed è importante saper spiegare *perché* si passa da un livello al successivo, non solo elencarli.

1. **Agente reattivo semplice:** decide guardando solo la percezione corrente, applicando regole del tipo “se percepisco X, faccio Y”. È semplicissimo da implementare, ma funziona bene solo se l'ambiente è completamente osservabile. Se l'agente non riesce a distinguere in che punto dello spazio si trova (percezione insufficiente), rischia di cadere in loop infiniti — per romperli serve introdurre un po' di casualità nel comportamento.
2. **Agente reattivo basato su modello:** quando l'osservabilità è solo parziale, la sola percezione corrente non basta più. L'agente allora mantiene uno **stato interno**, una rappresentazione di come il mondo evolve e di quali effetti hanno le proprie azioni. Questo gli permette di *prevedere* conseguenze anche quando il sensore, in un dato istante, non gli dà informazione sufficiente — tornando all'esempio della batteria, un modello che sa che “tre passi consumano una tacca” può prevedere il livello anche senza vederlo con precisione.
3. **Agente basato su obiettivi:** qui compare per la prima volta la nozione di *goal*. L'agente non si limita più a reagire, ma valuta le proprie azioni in funzione di quanto lo avvicinano a un obiettivo esplicito, eventualmente costruendo un piano di più passi tramite un *ragionamento ipotetico* (“se faccio questo, cosa succede?”). Il grande vantaggio, rispetto ai livelli precedenti, è che **basta cambiare l'obiettivo** per ottenere un comportamento completamente diverso, senza dover riscrivere nessuna regola — mentre in un agente reattivo il comportamento è “cablato” nelle regole stesse.
4. **Agente basato sull'utilità:** un goal è binario (raggiunto o no), ma spesso vogliamo distinguere fra più soluzioni che raggiungono comunque l'obiettivo. Qui entra una **funzione di utilità**, che valuta quanto è buona una soluzione tenendo conto di più fattori insieme (costo, tempo, rischio...). Il problema dell'agente diventa allora duplice: raggiungere l'obiettivo, e farlo massimizzando l'utilità.
5. **Agente che apprende:** aggiunge un ciclo di apprendimento vero e proprio, con tre componenti — un *Critico* che valuta la prestazione dell'agente confrontandola con uno standard esterno, un *Modulo di apprendimento* che modifica il comportamento sulla base di questo feedback, e un *Generatore di problemi* che suggerisce azioni esplorative (anche non ottimali nell'immediato) per esporre l'agente a nuove esperienze utili in futuro.

Perché conta: Questa scala di sofisticazione non è un elenco arbitrario: ogni livello risolve esattamente il limite del livello precedente. Il reattivo semplice fallisce quando l'osservabilità è parziale $\Rightarrow$ serve un modello. Il modello da solo non sa distinguere buone da cattive soluzioni $\Rightarrow$ serve un obiettivo. L'obiettivo da solo non distingue soluzioni ugualmente valide $\Rightarrow$ serve un'utilità. E nessuno dei livelli precedenti migliora nel tempo $\Rightarrow$ serve l'apprendimento. Se all'esame vi chiedono “perché serve X”, la risposta è quasi sempre “perché il livello precedente non riesce a gestire questo caso”.

1.4 Dal come al cosa: il paradigma dichiarativo

Un tema che attraversa il modulo (e in realtà tutto il corso) è la differenza tra programmazione *imperativa* e *dichiarativa*. Un programma tradizionale (pensate a una funzione C che inserisce un elemento in una lista ordinata) codifica esplicitamente, passo per passo, *come* ottenere il risultato: è una sequenza fissa di istruzioni pensata per un singolo compito.

Il software di IA, invece, separa una descrizione **dichiarativa** del problema (cosa sappiamo, cosa vogliamo) da un **motore generale** di inferenza o ricerca che sa elaborare quella descrizione. Il punto forte è che lo stesso motore generale può essere applicato a descrizioni diverse per risolvere problemi completamente diversi, senza essere riscritto.

L'esempio guida è il **mondo dei blocchi**: l'agente percepisce una configurazione iniziale di blocchi impilati, riceve un obiettivo (una configurazione finale desiderata), e deve costruire *da solo* — tramite ricerca nello spazio degli stati — la sequenza di mosse (Prendi, Impila, Metti) che porta dall'una all'altra. Il punto concettuale fondamentale, che vale la pena saper esprimere a parole proprie: **il programma dell'agente costruisce un programma**. Non esegue una soluzione già scritta dal programmatore, la genera lui stesso elaborando la conoscenza sulle azioni disponibili.

2 Ricerca nello spazio degli stati

2.1 L'idea di fondo

Moltissimi problemi si possono pensare come un insieme di **stati** collegati da **azioni**: risolvere il problema equivale a trovare un percorso che porti da uno stato iniziale a uno stato che soddisfa un obiettivo. Perché questa astrazione funzioni bene, di solito si assume che gli stati siano discreti, che le azioni abbiano effetto deterministico e che il mondo sia statico (cambia solo per effetto delle azioni dell'agente) — un principio guida importante è che si rappresenta *solo l'informazione rilevante* al problema: per uno stato conta la posizione, non il panorama.

Un problema di ricerca si formalizza sempre con quattro ingredienti, e vale la pena saperli elencare a memoria perché ogni esercizio parte da qui:

1. lo **stato iniziale**;
2. la **funzione successore** (le azioni possibili e il modello di transizione che dice a quale stato portano);
3. il **test obiettivo**;
4. la **funzione di costo del cammino**.

Un aspetto concettualmente delicato: lo **spazio degli stati** (l'insieme astratto di tutti gli stati e transizioni possibili, definito dal problema) è una cosa diversa dall'**albero (o grafo) di ricerca**, che è invece la struttura dati che l'algoritmo costruisce esplorando: ogni nodo dell'albero aggiunge informazione (genitore, azione, profondità, costo accumulato) allo stato che rappresenta. Diventa un grafo (anziché un albero) quando lo stesso stato è raggiungibile da più percorsi diversi.

Per confrontare le strategie di ricerca si usano sempre quattro criteri: **completezza** (trova sempre una soluzione se esiste?), **ottimalità** (trova quella di costo minimo?), e le due complessità, temporale e spaziale, misurate in numero di nodi generati/mantenuti (non in secondi o byte, perché un algoritmo è un'entità astratta). I parametri standard sono b (fattore di ramificazione), d (profondità della soluzione più superficiale) e m (profondità massima dello spazio, che può anche essere infinita).

2.2 Ricerca non informata: un confronto ragionato

Le strategie “blind” usano solo la struttura del problema, senza alcuna conoscenza aggiuntiva su quanto uno stato sia vicino al goal. La differenza fra loro sta tutta in *come gestiscono la frontiera* (l'insieme dei nodi generati ma non ancora espansi).

La **ricerca in ampiezza** (BFS) usa una coda FIFO ed esplora un livello di profondità alla volta: è completa e ottima (se i costi sono uniformi), ma ha una complessità spaziale $O(b^{d+1})$ devastante in pratica, perché deve tenere in memoria tutta la frontiera e tutti i suoi antenati — con $b = 10$ e $d = 12$ servirebbero circa 10 petabyte.

La **ricerca in profondità** (DFS) usa invece uno stack: scende sempre lungo un ramo fino in fondo prima di tornare indietro. Il vantaggio enorme è la memoria, $O(bm)$ (o perfino $O(m)$ se si genera un successore alla volta), ma paga il prezzo di non essere né completa (può perdersi in un ramo infinito) né ottima (può trovare per prima una soluzione più lunga di un'altra, semplicemente perché l'ha incontrata scendendo per quel ramo).

L'idea più elegante per unire i pregi di entrambe è l'**iterative deepening** (IDDFS): si esegue ripetutamente una DFS a profondità limitata, aumentando il limite $0, 1, 2, \dots$ fino a trovare la soluzione, ricostruendo l'albero da zero ad ogni iterazione. Sembra spreco puro rigenerare

l'albero ogni volta, ma non lo è: la maggior parte dei nodi si trova vicino alla frontiera (ai livelli più profondi), quindi i nodi vicini alla radice vengono rigenerati molte volte ma sono relativamente pochi. Il risultato è che l'IDDFS eredita la completezza e l'ottimalità della BFS con la parsimonia di memoria della DFS, $O(bd)$ — è per questo che viene presentata come **la scelta preferita** quando lo spazio è ampio e non si conosce a priori la profondità della soluzione.

Quando i costi delle azioni non sono tutti uguali, la BFS da sola non basta più (contare i passi non equivale più a contare il costo): serve la **ricerca a costo uniforme**, che usa una coda a priorità ordinata sul costo accumulato $g(n)$ ed espande sempre il cammino più economico aperto. Un dettaglio da ricordare: quando si genera per la prima volta un nodo obiettivo, l'algoritmo *non si ferma subito* — deve prima verificare che non ci siano altri cammini ancora aperti più economici.

2.3 Ricerca informata: perché la conoscenza aiuta

Le strategie non informate esplorano “alla cieca”. Se disponiamo di una **funzione euristica** $h(n)$ — una stima di quanto costi arrivare dal nodo n a un obiettivo — possiamo usarla per ordinare la frontiera e concentrare la ricerca sulle direzioni più promettenti, riducendo drasticamente il lavoro sprecato.

La **ricerca greedy** usa $f(n) = h(n)$: guarda solo avanti, ignorando quanto si è già speso per arrivare a n . Il difetto è che eredita i vizi della DFS — se l'euristica inganna (una direzione sembra promettente ma porta in un vicolo cieco), la ricerca può fare grandi giri a vuoto: non è né completa né ottima in generale.

L'idea vincente è **A***, che combina il guardare indietro della ricerca a costo uniforme con il guardare avanti della greedy:

$$f(n) = g(n) + h(n)$$

dove $g(n)$ è il costo (esatto, osservato) per arrivare da inizio a n , e $h(n)$ è la stima del costo residuo. L'intuizione è che $f(n)$ stima il costo totale di un cammino risolutivo che passa per n .

Perché A* funzioni correttamente, l'euristica deve essere **ammissibile**: non deve mai sovrastimare il costo reale,

$$h(n) \leq h^*(n) \quad \text{per ogni nodo } n,$$

dove $h^*(n)$ è il costo reale (ma sconosciuto) del cammino ottimo residuo. **Se l'euristica è ammissibile e i costi dei passi sono strettamente positivi, A* è completo e ottimo** su un albero di ricerca. L'intuizione della dimostrazione è semplice e vale la pena saperla riformulare a parole: se esiste un nodo n sul cammino ottimo, il suo $f(n)$ non può mai superare C^* (il costo della soluzione ottima), mentre qualsiasi obiettivo raggiunto per un cammino subottimo avrebbe f maggiore di C^* — quindi A*, che espande sempre il nodo con f minima, sceglierà sempre n prima di un obiettivo peggiore.

Su un **grafo**, dove uno stesso stato può essere raggiunto da più cammini, l'ammissibilità da sola non basta più: serve la proprietà più forte di **consistenza (o monotonicità)**,

$$h(n) \leq c(n, a, n') + h(n'),$$

una vera e propria disuguaglianza triangolare, che garantisce che f non decresca mai lungo un cammino — e quindi che il primo obiettivo *espanso* (non solo generato) sia già quello ottimo. Ogni euristica consistente è anche ammissibile, ma non vale il viceversa.

A* è inoltre **ottimamente efficiente**: nessun altro algoritmo, a parità di euristica, può garantire di espandere meno nodi. Il difetto pratico però è che mantiene in memoria *tutti* i nodi

generati (si comporta come una BFS sotto quel profilo), il che lo rende spesso impraticabile per limiti di memoria. Per questo esistono varianti come **RBFS**, che si comporta come una DFS ricorsiva ma tiene traccia di un “upper bound” dinamico (il valore della migliore alternativa nota fra i fratelli): appena un ramo smette di essere il più promettente, viene abbandonato — a costo di dover talvolta rifare lavoro già svolto se in seguito quel ramo torna conveniente. Il vantaggio è una memoria lineare, $O(bd)$.

2.3.1 Da dove vengono le buone euristiche

Un modo sistematico (e molto elegante) per costruire euristiche ammissibili è pensare a un **problema rilassato**: se togliamo dei vincoli al problema originale, otteniamo un problema più facile il cui grafo degli stati contiene comunque tutte le soluzioni ottime originali (più altre in aggiunta). Ne segue che il costo della soluzione ottima nel problema rilassato *non può mai* essere superiore al costo reale del problema originale — è quindi automaticamente un’euristica ammissibile.

L’esempio classico è l’8-puzzle: il vincolo vero è che una tessera si sposta in una cella adiacente *vuota*. Se rilassiamo solo il vincolo “la cella deve essere vuota” (permettendo di scambiare tessere adiacenti anche occupate), otteniamo un’euristica vicina alla **distanza di Manhattan** (h_2); se rilassiamo anche il vincolo di adiacenza (una tessera può andare ovunque), otteniamo l’euristica “**tessere fuori posto**” (h_1). Sperimentalmente e teoricamente h_2 è migliore di h_1 — si dice che **domina** h_1 , perché $h_2(n) \geq h_1(n)$ per ogni n , e questo garantisce che A^* con h_2 espanda un sottoinsieme dei nodi espansi con h_1 . Per misurare quanto un’euristica sia informativa in pratica si usa il **fattore di ramificazione effettivo** b^* , definito implicitamente da

$$N + 1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d,$$

dove N è il numero di nodi generati e d la profondità della soluzione trovata: più b^* è vicino a 1, più l’euristica guida bene la ricerca.

Perché conta: Un modo rapido per ricordarsi come si incastrano tutte le strategie viste in questo modulo: A^* con $h(n) = 0$ degenera esattamente nella ricerca a costo uniforme (e, se i costi sono uniformi, nella BFS). La ricerca informata non è quindi un algoritmo diverso da quella non informata, è la *stessa* famiglia di algoritmi con in più un termine che guida la scelta. Se all’esame vi chiedono “che relazione c’è fra A^* e la ricerca a costo uniforme”, la risposta è proprio questa.

3 Ricerca con avversario: i giochi

3.1 Perché i giochi interessano l'IA

Marvin Minsky diceva che i giochi non si studiano perché sono semplici, ma perché offrono la massima complessità con le minime strutture iniziali: regole chiarissime, ma spazio di ricerca enorme, e soprattutto la presenza di un **avversario** — un elemento che rompe l'idea di “ricerca in solitaria” vista nel modulo precedente. In un gioco, l'evoluzione dello stato non è più controllata da un solo agente: bisogna ragionare ipoteticamente anche sulle mosse altrui.

Il modulo si concentra su giochi **deterministici**, a **informazione completa** (tutto lo stato è visibile, come a scacchi), a **due giocatori** e a **somma zero** — cioè situazioni in cui il guadagno di un giocatore è esattamente compensato dalla perdita dell'altro. Questa proprietà è quello che rende tutto più semplice da rappresentare: se l'utilità di uno stato terminale per il giocatore A vale u , per B vale $-u$, quindi basta memorizzare un solo numero per nodo per descrivere entrambi i punti di vista.

Rispetto alla ricerca “classica”, qui cambia tutto l'obiettivo: non cerchiamo un semplice cammino a costo minimo, ma una **strategia** — una funzione che associa una mossa a ogni possibile stato raggiungibile, perché non sappiamo in anticipo cosa farà l'avversario. Per costruire questa strategia si adotta un'**ipotesi di pessimismo**: si assume che l'avversario sia infallibile e giochi sempre la mossa a lui più vantaggiosa (cioè la peggiore per noi). È un'ipotesi conservativa, ma garantisce una strategia sicura anche contro l'avversario più forte immaginabile.

3.2 Minimax

L'algoritmo **Minimax** formalizza esattamente questa idea. Si costruisce (concettualmente) l'intero albero di gioco, con i nodi che si alternano fra turno di MAX (che vuole massimizzare) e turno di MIN (l'avversario, che vuole minimizzare il valore per MAX). Il valore di ogni nodo si calcola ricorsivamente dal basso verso l'alto:

$$\text{Minimax}(n) = \begin{cases} \text{Utilità}(n) & \text{se } n \text{ è terminale} \\ \max_{s \in \text{Succ}(n)} \text{Minimax}(s) & \text{se } n \text{ è un nodo MAX} \\ \min_{s \in \text{Succ}(n)} \text{Minimax}(s) & \text{se } n \text{ è un nodo MIN} \end{cases}$$

Il nome “minimax” racconta esattamente la logica: minimizzando il massimo guadagno possibile dell'avversario, ciascun giocatore minimizza automaticamente la propria perdita peggiore. Il valore trovato alla radice non è il massimo assoluto raggiungibile nell'albero (per raggiungere quello, MAX dovrebbe sperare in un errore di MIN, cosa che l'ipotesi di pessimismo esclude), ma è il massimo risultato che MAX può *garantirsi* contro un avversario ottimale.

Minimax è completo (su alberi finiti) e ottimo, ma solo se entrambi i giocatori giocano in modo ottimale; se l'avversario reale gioca peggio, la strategia resta comunque *sicura* — garantisce almeno il valore calcolato. Il costo però è quello di una visita esaustiva: tempo $O(b^m)$, spazio $O(bm)$ (lineare, come tutte le DFS).

Perché conta: Vale la pena collegare Minimax alla teoria delle decisioni generale: quando un esito dipende da un fattore non controllabile, si può essere ottimisti (**maximax**, massimizzare il caso migliore), pessimisti/conservativi (**maximin**, massimizzare il caso peggiore) oppure minimizzare il rimpianto rispetto alla scelta che, col senno di poi, sarebbe stata ottimale (**minimax regret**). Minimax nei giochi altro non è che un maximin applicato ricorsivamente lungo tutto l'albero — l'avversario gioca il ruolo del “fattore non controllabile”.

3.3 Potatura alfa-beta: rendere Minimax praticabile

Il problema pratico di Minimax è che $O(b^m)$ è del tutto improponibile in giochi reali come gli scacchi. L'osservazione chiave dell'**alfa-beta pruning** è che non serve esplorare tutti i rami: se durante la visita si scopre che un ramo non potrà mai cambiare la decisione alla radice (perché è già “battuto” da un'alternativa già trovata altrove), possiamo evitare di espanderlo senza perdere nulla in termini di correttezza.

Si mantengono due valori lungo la ricorsione:

- α : il valore della miglior scelta trovata finora per MAX lungo il cammino (parte da $-\infty$, aggiornato nei nodi MAX);
- β : il valore della miglior scelta trovata finora per MIN (parte da $+\infty$, aggiornato nei nodi MIN).

Vale la pena continuare a esplorare un nodo solo finché il suo valore resta nell'intervallo $[\alpha, \beta]$: appena $\beta \leq \alpha$, nessun valore rimanente in quel sottoalbero potrà mai influenzare la decisione finale, e si può potare. Il risultato notevole è che l'alfa-beta trova *sempre esattamente* la stessa mossa e lo stesso valore di Minimax, ma esplorando molti meno nodi: nel caso migliore la complessità scende da $O(b^m)$ a $O(b^{m/2})$ — dimezzare l'esponente vuol dire, a parità di tempo, poter guardare il doppio della profondità.

C'è però un dettaglio cruciale: questo guadagno dipende **fortemente dall'ordine** in cui si esaminano le mosse. Se la mossa migliore (la “killer move”) viene esaminata per ultima, i bound α e β restano larghi per tutta l'esplorazione precedente e la potatura non scatta quasi mai — nel caso peggiore si degenera esattamente in Minimax puro, $O(b^m)$. Con un ordinamento casuale (il caso medio realistico) si ottiene $O(b^{3m/4})$; con un ordinamento ottimale si arriva al caso migliore $O(b^{m/2})$. Ecco perché in pratica si combinano tecniche per anticipare le mosse promettenti: apprendimento dall'esperienza passata, iterative deepening (le informazioni ottenute a profondità minori orientano l'ordinamento a profondità maggiori), e tabelle di trasposizione (una hash table che riconosce quando sequenze di mosse diverse portano allo stesso stato, evitando di riesplorarlo).

3.4 Quando non si può guardare fino in fondo

In giochi complessi non è possibile arrivare fino agli stati terminali entro un tempo ragionevole. Si introduce allora un **test di taglio** che interrompe la ricerca a una certa profondità e restituisce una stima invece del valore esatto, calcolata con una **funzione di valutazione euristica** — tipicamente una combinazione lineare di feature pesate, $\text{eval}(s) = \sum_i w_i f_i(s)$ (ad esempio, agli scacchi, il numero di pedoni, alfieri, torri ancora in gioco).

Tagliare artificialmente la ricerca introduce però il **problema dell'orizzonte**: la valutazione può essere instabile proprio nel punto in cui si taglia, rischiando di non vedere un ribaltamento imminente della partita. La soluzione concettuale (Berliner, 1973) è la nozione di **quiescenza**: si tagliano solo i nodi la cui valutazione è stabile nel tempo, mentre i nodi “non quiescenti” vengono esplorati oltre il limite prestabilito.

Due sistemi storici mostrano bene come questi principi si combinano nella pratica: **DeepBlue** (scacchi, 1996, prima macchina a battere un campione del mondo in carica) usava iterative deepening più alfa-beta più tabelle di trasposizione; **AlphaGo** (Go, 2015) combinava reti neurali profonde con la ricerca ad albero, addestrandosi sia su partite umane sia per *self-play*.

4 Constraint Satisfaction Problems (CSP)

4.1 Perché aprire la scatola nera

Nei moduli precedenti uno stato era una “scatola nera”: l’unica cosa che potevamo fare era generare successori e testare l’obiettivo. Nei CSP, invece, **apriamo la scatola**: uno stato ha una struttura interna esplicita, fatta di variabili con valori, e i vincoli fra le variabili sono rappresentati in modo dichiarativo. Questo permette di costruire algoritmi generali (non specifici per un singolo problema) e tecniche di potatura molto più efficaci della ricerca cieca.

Un CSP è definito da un insieme di **variabili** $X_1, \dots, X_n$, un insieme di **domini** $D_1, \dots, D_n$ (i valori ammissibili per ciascuna) e un insieme di **vincoli** $C_1, \dots, C_m$. Un assegnamento è **completo** se dà un valore a tutte le variabili, **consistente** se non viola alcun vincolo, e una **soluzione** è esattamente un assegnamento completo e consistente. L’esempio guida delle slide è la colorazione della mappa dell’Australia: sette territori, tre colori, vincoli binari del tipo “territori confinanti devono avere colori diversi”.

I vincoli possono essere unari (una sola variabile), binari (rappresentabili come archi di un grafo dei vincoli — questa rappresentazione è centrale per gli algoritmi che vedremo) o n-ari (rappresentabili come ipergrafi, es. il vincolo *alldifferent*, o l’equazione globale di un puzzle crittoaritmico come SEND+MORE=MONEY).

4.2 Il trucco decisivo: la commutatività

Un CSP si può formulare come problema di ricerca (stato iniziale = assegnamento vuoto, successore = assegna un valore a una variabile, goal = assegnamento completo), ma con una proprietà speciale che lo rende molto più trattabile della ricerca generica: **l’ordine con cui si assegnano le variabili non conta ai fini del risultato finale**. Assegnare prima X_1 poi X_2 oppure viceversa porta allo stesso identico stato $\{X_1 = v_1, X_2 = v_2\}$. Questa proprietà, chiamata **commutatività**, permette agli algoritmi di fissare sempre prima *una* variabile e poi *un* valore per essa, senza dover trattare come rami distinti tutti i possibili ordini delle variabili — questo riduce il fattore di ramificazione da $n \cdot d$ (con $n! \cdot d^n$ foglie possibili) a semplicemente d per livello (d^n foglie): nell’esempio dell’Australia, si passa da oltre 11 milioni di foglie a sole 2187.

Il **generate-and-test** (genera un assegnamento completo, poi controllalo) è comunque troppo costoso: con 8 regine rappresentate ingenuamente si arriva a $64^8 \approx 2.8 \times 10^{14}$ possibilità. Il **backtracking** (una DFS che usa i vincoli per potare i rami appena si genera un conflitto, invece di aspettare l’assegnamento completo) è molto meglio, ma da solo soffre di **thrashing**: può ripetere identico lo stesso errore in punti diversi dell’albero, perché non “ricorda” perché ha fallito la volta precedente.

4.3 Euristiche: dove rischiare e come minimizzare il rischio

Per migliorare il backtracking, ci si pone tre domande, e ciascuna ha una risposta elegante:

Quale variabile scegliere per prima? L’euristica **MRV** (Minimum Remaining Values, detta anche “fail-first”) sceglie la variabile con *meno* valori legali residui, per scoprire il prima possibile i vicoli ciechi invece che tardi, dopo aver già investito lavoro. Quando MRV non aiuta a discriminare (ad esempio all’inizio, quando nessun vincolo è ancora stato attivato), si usa come spareggio la **degree heuristic**: si sceglie la variabile coinvolta nel maggior numero di vincoli con variabili ancora non assegnate.

Quale valore provare per primo? Qui la logica si ribalta: l’euristica **LCV** (Least Constraining Value) sceglie il valore che lascia *più* opzioni residue ai vicini, per massimizzare la probabilità di successo del ramo che stiamo esplorando.

Perché conta: Non è affatto contraddittorio usare “più vincolata” per la variabile e “meno vincolante” per il valore: rispondono a due domande diverse. Per la variabile, vogliamo scoprire *dove* rischiamo di fallire il prima possibile (fail-first). Per il valore, una volta lì, vogliamo *minimizzare* il rischio di fallire scegliendo l’opzione che lascia più margine di manovra agli altri. Se all’esame vi chiedono di spiegare questa apparente contraddizione, è proprio questa la risposta.

4.4 Propagazione dei vincoli: guardare avanti prima di sbagliare

Le euristiche di ordinamento migliorano *come* esploriamo, ma non riducono da sole lo spazio di ricerca. Il passo successivo è alternare esplorazione e **inferenza**: dopo ogni assegnamento, si propagano le conseguenze sui domini delle altre variabili.

Il livello più economico è il **forward checking**: appena si assegna una variabile, si eliminano subito dai domini dei vicini diretti i valori ormai incompatibili. È economico e si abbina naturalmente a MRV (perché mantiene sempre aggiornato il numero di valori residui). Il suo limite, però, è che non controlla le conseguenze fra due vicini fra loro non ancora assegnati: se WA e Q sono già stati fissati e riducono sia NT sia SA allo stesso, identico valore residuo, il forward checking non se ne accorge — anche se NT e SA sono confinanti, e quindi quell’assegnamento è già di fatto impossibile.

Un livello più forte è l’**arc consistency**, realizzata dall’algoritmo **AC-3** (Mackworth, 1977): un arco $X \rightarrow Y$ è consistente se per ogni valore di X esiste almeno un valore compatibile in Y . L’algoritmo mantiene una coda di archi da verificare; ogni volta che si elimina un valore dal dominio di una variabile, si rimettono in coda tutti gli archi che puntano *verso* di essa, perché quella riduzione potrebbe aver reso quegli archi non più consistenti — è così che l’informazione si propaga “a catena” su tutto il grafo, non solo sui vicini immediati come nel forward checking. Il costo è $O(n^2d^3)$, più alto del forward checking ma anche più efficace. Attenzione però: AC-3 è **incompleto** — un grafo può essere completamente arc-consistent e comunque non avere nessuna soluzione (l’esempio classico è un triangolo di tre variabili, tutte vincolate a essere diverse a due a due, con solo due colori disponibili: localmente ogni arco è consistente, ma globalmente non esiste un modo di colorare tutte e tre le variabili). Per catturare anche questo tipo di inconsistenza servirebbe la **path consistency**, che ragiona su triplete di variabili — più forte, ma anche più costosa.

4.5 Oltre il backtracking cronologico

Il backtracking “standard” torna sempre indietro alla variabile assegnata immediatamente prima, anche se quella variabile non ha nulla a che fare con il conflitto trovato — uno spreco evidente. Il **backjumping** usa invece un **conflict set** (l’insieme degli assegnamenti che hanno effettivamente causato il fallimento) per saltare direttamente alla variabile davvero responsabile. Il **conflict-directed backjumping** raffina ulteriormente questa idea, aggiornando dinamicamente i conflict set man mano che si esplora — di fatto imparando dai propri fallimenti, in modo simile al *clause learning* dei moderni SAT solver.

5 Logica proposizionale e rappresentazione della conoscenza

5.1 Perché serve un linguaggio formale

Rappresentare la conoscenza vuol dire dotarsi di un linguaggio in cui esprimere ciò che sappiamo del mondo, così da poter applicare processi di ragionamento *automatici*. Ragionare, in questo senso, vuol dire rendere esplicita conoscenza che era implicita: se so che tutti i cetacei vivono in mare e che tutte le balene sono cetacei, posso concludere che tutte le balene vivono in mare, senza aver verificato personalmente ogni singola balena — mi basta assumere vere le premesse e applicare uno schema di ragionamento valido.

Il punto cruciale, e la ragione per cui questo è automatizzabile, è che questo tipo di ragionamento lavora **sulla forma** delle affermazioni (schemi astratti), non sul loro contenuto specifico. Un agente basato sulla conoscenza è definito da una **Knowledge Base (KB)**, da un'operazione **tell** per aggiungere formule e da un'operazione **ask** per interrogarla — con il vincolo, sempre valido, che ogni risposta a una **ask** deve essere conseguenza logica di quanto è stato asserito.

5.2 I concetti cardine: modello, conseguenza, inferenza

Un **modello** è semplicemente un “mondo possibile” che fissa il valore di verità di ogni formula. Diciamo che $A \models B$ (“ B è conseguenza logica di A ”) quando B è vera in *tutti* i modelli in cui A è vera — in termini insiemistici, l'insieme dei modelli di A è contenuto nell'insieme dei modelli di B . Attenzione alla direzionalità: $A \not\models B$ significa solo che *esiste almeno un* controesempio, non che B sia sempre falsa quando A è vera.

Una formula è **valida** (tautologia) se è vera in ogni modello, **insoddisfacibile** (contraddizione) se è falsa in ogni modello, e **soddisfacibile** se è vera in almeno un modello — quest'ultima nozione è esattamente ciò che cerchiamo quando risolviamo un CSP: un modello che soddisfi tutti i vincoli.

L'**inferenza** è invece un processo **sintattico**: lavora sulla struttura delle formule secondo regole del linguaggio, non sul loro significato. Un buon algoritmo di inferenza deve sempre essere **corretto** (soundness: se deriva qualcosa, quel qualcosa è davvero conseguenza logica — questo non si può mai sacrificare) e idealmente **completo** (completeness: se qualcosa è conseguenza logica, l'algoritmo riesce a derivarlo — questo non è sempre garantito).

5.3 Sintassi e semantica proposizionale

Con N simboli proposizionali abbiamo 2^N modelli possibili. L'operatore su cui vale la pena soffermarsi è l'**implicazione**, perché è quello che genera più confusione:

$$P \Rightarrow Q \equiv \neg P \vee Q.$$

Se P è falsa, l'implicazione è vera “per vacuità”, indipendentemente da Q ; se P è vera, allora Q deve necessariamente esserlo. Il punto da tenere bene a mente è che l'**implicazione logica non è una relazione causale**: dipende solo dai valori di verità, non dal significato delle due proposizioni. L'esempio didattico classico: “Torino è in Lombardia $\Rightarrow$ Giulio Cesare governò Roma” è un'implicazione *vera*, non perché ci sia un nesso causale (non c'è nessun nesso!), ma semplicemente perché l'antecedente è falso (Torino è in Piemonte) — e un antecedente falso rende vera l'implicazione a prescindere dal conseguente.

5.4 Come verificare se KB implica qualcosa

Ci sono due strade. La prima, il **model checking**, enumera tutti i 2^N modelli — corretta ma esponenziale, quindi impraticabile per KB grandi. La seconda, il **theorem proving**, cerca una derivazione sintattica senza costruire esplicitamente i modelli.

La tecnica più elegante è la **dimostrazione per refutazione**: per verificare $KB \models P$, si assume per assurdo $\neg P$, lo si aggiunge alla KB, e si dimostra che $KB \wedge \neg P$ è insoddisfacibile (cioè che se ne può derivare **False**) — esattamente il ragionamento per assurdo che già conoscete dalla matematica.

Per applicare questa idea servono due ingredienti: portare tutto in **forma normale congiuntiva** (CNF, una congiunzione di clausole, ciascuna disgiunzione di letterali), e poi applicare la **regola di risoluzione**: da due clausole che condividono un letterale complementare (una lo nega, l'altra no), si produce una nuova clausola con tutti i letterali tranne quella coppia. Il modus ponens, che già conoscete, è semplicemente un caso particolare di risoluzione. L'algoritmo di risoluzione, applicato ripetutamente, cerca di produrre la **clausola vuota** (cioè **False**): se ci riesce, $KB \models P$ è confermato. Questa procedura è **corretta e completa**: è una delle poche tecniche di IA per cui si può dimostrare rigorosamente che funziona sempre, se le si dà abbastanza tempo.

5.5 Clausole di Horn: quando l'inferenza diventa veloce

In moltissimi contesti pratici, la conoscenza si presenta in una forma molto particolare: le **clausole di Horn**, disgiunzioni in cui *al più un* letterale è positivo. Perché sono così importanti? Perché catturano esattamente le implicazioni con antecedente congiuntivo ($\neg A \vee \neg B \vee C \equiv A \wedge B \Rightarrow C$), e su di esse si può fare inferenza in **tempo lineare** rispetto alla dimensione della KB — molto più efficiente della risoluzione generale, ed è per questo che sono la base della programmazione logica (Prolog).

Su clausole di Horn esistono due strategie complementari:

- **Forward chaining**: parte dai fatti noti e applica ripetutamente il modus ponens, aggiungendo man mano nuovi fatti derivati, finché non deriva la query. È *data-driven*: guidato solo da ciò che sappiamo, senza sapere dove vogliamo arrivare. È completo e lineare, ma può fare inferenze del tutto irrilevanti alla domanda specifica.
- **Backward chaining**: parte invece dal goal e cerca ricorsivamente le regole che lo concludono, dimostrandone le premesse. È *goal-driven*: sa esattamente cosa sta cercando di dimostrare, e per questo è generalmente più efficiente (più focalizzato) — è la strategia usata da Prolog e nel theorem proving.

Perché conta: Notate lo schema che si ripete in tutto il modulo: la correttezza (soundness) di un algoritmo di inferenza non si può mai sacrificare, perché è ciò che garantisce che il sistema non produca mai conclusioni false; la completezza invece si può “comprare” pagando in efficienza (risoluzione generale, completa ma costosa) o restringendo la forma della KB (clausole di Horn, veloci ma non tutte le KB si lasciano tradurre così). Questo compromesso fra espressività, completezza ed efficienza è lo stesso identico tema che ritroverete, ancora più marcato, nel modulo sulla logica del primo ordine.

6 Logica del primo ordine (FOL)

6.1 Il limite della proposizionale

La logica proposizionale è dichiarativa, composizionale e non ambigua — tre proprietà preziose che vogliamo conservare. Ma ha un difetto grave: non è **espressiva** abbastanza. Non permette rappresentazioni compatte (per dire “chi è felice ascolta musica” dovremmo scrivere una regola per ogni singola persona) e non permette di parlare di **relazioni fra oggetti** — ogni fatto è un simbolo atomico indivisibile.

La logica del primo ordine risolve questo problema introducendo **oggetti** in **relazione** fra loro: costanti (che nominano oggetti specifici), predicati (che esprimono proprietà o relazioni, restituendo vero/falso), funzioni (che restituiscono un oggetto — attenzione, una funzione non *costruisce* l’oggetto, lo *riferisce*: “**GambaSinistra**(John)” indica un oggetto che già esiste nel dominio, senza bisogno di sapere come sia fatta una gamba), e i due quantificatori $\forall$ e $\exists$.

Un modello FOL è una coppia $M = (D, I)$: un dominio D (l’insieme degli oggetti) e un’interpretazione I (che collega i simboli del linguaggio agli elementi/relazioni del dominio). Il valore di verità di una formula **dipende sempre dal modello scelto** — non è mai assoluto. E qui c’è una complicazione seria rispetto alla proposizionale: con un dominio potenzialmente illimitato, i modelli possibili possono essere *infiniti*, il che rende impraticabile l’enumerazione dei modelli che funzionava in proposizionale.

6.2 L’errore più comune: abbinare male i quantificatori

Questo è probabilmente il punto in cui vale più la pena investire tempo, perché è la fonte più frequente di errore.

- $\forall x F$ è vera se F vale per *ogni* oggetto del dominio.
- $\exists x F$ è vera se F vale per *almeno un* oggetto del dominio.

La regola pratica da interiorizzare è: $\forall$ **va naturalmente con** $\Rightarrow$, mentre $\exists$ **va naturalmente con** $\wedge$. Perché? Consideriamo “tutti i partecipanti al corso sono intelligenti”. Scritta come

$$\forall x \text{Partecipa}(x, \text{SISINT}) \Rightarrow \text{Intelligente}(x)$$

dice esattamente questo: per ogni oggetto del dominio (comprese le sedie, i numeri, tutto), *se* partecipa al corso, allora è intelligente — e per tutti gli oggetti che non partecipano, l’implicazione è vera per vacuità, quindi non “sporca” l’affermazione. Se scrivessimo invece

$$\forall x \text{Partecipa}(x, \text{SISINT}) \wedge \text{Intelligente}(x)$$

staremmo affermando che *ogni singolo oggetto del dominio* partecipa al corso *e* è intelligente — un’affermazione quasi certamente falsa (le sedie non partecipano al corso).

Simmetricamente, per dire “esiste qualcuno intelligente fra i partecipanti” dobbiamo scrivere

$$\exists x \text{Partecipa}(x, \text{SISINT}) \wedge \text{Intelligente}(x),$$

mentre

$$\exists x \text{Partecipa}(x, \text{SISINT}) \Rightarrow \text{Intelligente}(x)$$

è tecnicamente vera anche se **nessun** partecipante è intelligente, perché basta che esista un qualsiasi oggetto che *non* partecipa al corso (una sedia, di nuovo) per rendere l’implicazione vera per vacuità su quell’oggetto. È un errore ancora più insidioso del precedente, perché la formula “sembra” dire la cosa giusta ma non la dice affatto.

6.3 L'ordine dei quantificatori conta

Quando due quantificatori dello stesso tipo si susseguono, l'ordine è irrilevante ($\forall x \forall y \equiv \forall y \forall x$). Ma quando i tipi sono diversi, l'ordine cambia completamente il significato:

$$\forall x \exists y \text{ Ama}(x, y) \quad \text{vs.} \quad \exists y \forall x \text{ Ama}(x, y).$$

La prima dice che ogni persona ama qualcuno (ma persone diverse possono amare persone diverse — ogni x “sceglie” il proprio y); la seconda dice che esiste una *singola* persona amata da *tutti*. La seconda è molto più forte e implica la prima, ma non viceversa. Questo concetto tornerà cruciale quando parleremo di skolemizzazione.

6.4 Dall'inferenza al lifting

In FOL possiamo procedere per **proposizionalizzazione**: istanziamo un $\forall x \alpha$ con qualsiasi termine ground del vocabolario (regola **UI**, che produce una KB logicamente equivalente), e istanziamo un $\exists x \alpha$ con *una sola* costante nuova (regola **EI**, la cosiddetta *skolemizzazione*, che produce una KB equivalente solo dal punto di vista inferenziale, non logico). Il **teorema di Herbrand** garantisce che, se una formula è davvero conseguenza logica, esiste sempre una dimostrazione finita — ma se *non* lo è, la ricerca può proseguire all'infinito senza mai terminare (per via delle funzioni, che possono generare infiniti termini ground). Questo rende FOL **semidecidibile**: possiamo sempre confermare un “sì”, ma non esiste un algoritmo generale in grado di confermare un “no”.

La proposizionalizzazione, però, è inefficiente: genera un'istanza per ogni costante del vocabolario, anche quando è ovvio a colpo d'occhio che la maggior parte sono inutili. La soluzione più efficiente è il **lifting**: si sollevano le regole di inferenza direttamente al primo ordine, usando **variabili** e **unificazione** invece di istanziare tutto esplicitamente. Il **Modus Ponens Generalizzato** ne è l'esempio principale: da n premesse atomiche e un'implicazione $p_1 \wedge \dots \wedge p_n \Rightarrow q$, se esiste una sostituzione θ che unifica ciascuna premessa con la corrispondente p_i , si conclude $q\theta$.

L'**unificazione** $\text{UNIFY}(F_1, F_2) = \theta$ trova la sostituzione che rende due formule sintatticamente identiche (idealmente il *Most General Unifier*); può fallire per conflitti fra variabili, risolvibili rinominando le variabili di formule diverse prima di confrontarle (standardizzazione separata).

6.5 Skolemizzazione: l'errore da non fare mai

Quando traduciamo FOL in CNF per applicare la risoluzione, dobbiamo eliminare i quantificatori esistenziali. Se un $\exists y$ compare *dentro* lo scope di un $\forall x$ esterno, **non** possiamo sostituirlo con una singola costante fissa: significherebbe affermare che *tutti* gli x condividono lo stesso valore di y , il che quasi sempre è falso. L'esempio più chiaro: “ogni persona abita in un luogo” è

$$\forall x \exists y \text{ Persona}(x) \Rightarrow \text{Abita}(x, y).$$

Sostituire $\exists y$ con una costante fissa K direbbe che *tutte le persone abitano nello stesso posto* — palesemente sbagliato. La soluzione corretta è introdurre una **funzione di Skolem** i cui argomenti sono tutte le variabili universali nel cui scope ricade l'esistenziale:

$$\forall x \text{ Persona}(x) \Rightarrow \text{Abita}(x, \text{luogo}(x)),$$

in modo che il luogo possa variare correttamente da persona a persona. Solo se l'esistenziale *non* ricade nello scope di alcun universale la funzione di Skolem degenera in una semplice costante (esattamente il caso della regola EI).

Con la risoluzione “liftata” a FOL (risoluzione binaria + fattorizzazione con unificazione al posto dell’uguaglianza sintattica) otteniamo una procedura **refutation-complete**: non enumera tutte le conseguenze logiche, ma se $KB \wedge \neg Q$ è davvero insoddisfacibile, la risoluzione trova sempre, in un numero finito di passi, la clausola vuota.

Perché conta: La semidecidibilità di FOL non è un dettaglio tecnico marginale: è la ragione profonda per cui i sistemi di inferenza reali (Prolog compreso) preferiscono il **backward chaining** al forward chaining generico, e per cui si insiste tanto sulle clausole di Horn/DATALOG (dove invece la terminazione è garantita). Se all’esame vi chiedono “perché non si usa sempre la risoluzione generale al posto del backward chaining”, il filo del ragionamento è: espressività maggiore $\Rightarrow$ rischio di non terminazione $\Rightarrow$ si preferisce restringere il linguaggio quando possibile.

7 Ontologie e Pianificazione automatica

7.1 Dalla tassonomia all'ontologia

Una **tassonomia** è un'organizzazione gerarchica di concetti costruita esclusivamente con la relazione **Is-a** (sottoclasse): se **PalloneCalcio** è sottoclasse di **Pallone**, ogni istanza della prima è anche istanza della seconda. Il vantaggio pratico è enorme: le istanze **ereditano** le proprietà delle superclassi, quindi non dobbiamo ripetere la stessa regola per ogni sottoclasse — evitiamo ridondanza e otteniamo inferenza automatica di proprietà mai dichiarate esplicitamente.

Accanto a Is-a esiste la relazione **Part-of** (transitiva: se X è parte di Y e Y è parte di Z , allora X è parte di Z), che descrive la composizione strutturale di un oggetto, invece della sua categorizzazione.

Un'ontologia si divide sempre in due parti: la **T-box** (le definizioni generali, lo “schema” del dominio — cosa può esistere e come si relaziona) e la **A-box** (i fatti concreti su istanze specifiche, coerenti con la T-box) — un po' come lo schema di un database rispetto ai suoi record effettivi.

Perché una tassonomia non basta sempre? Due problemi tipici: le **eccezioni al default** (gli uccelli di solito volano, ma pinguini e struzzi no — una sottoclasse può cancellare una proprietà altrimenti ereditata), e la **polisemia** (la stessa parola, come “cane”, può denotare concetti completamente diversi appartenenti ad alberi tassonomici distinti — un animale, una costellazione, un dente meccanico — e il motore inferenziale non deve mischiarli).

Un'ontologia generalizza la tassonomia: da un albero (solo relazioni Is-a) a un **grafo**, in cui possono coesistere Is-a, Part-of e relazioni di dominio arbitrarie (come “haMoglie”). Ogni tassonomia è quindi un caso particolare di ontologia, ma non vale il viceversa. Su queste rappresentazioni si costruisce il **Semantic Web** (RDF, con le sue triple soggetto-predicato-oggetto identificate univocamente da IRI; OWL 2, che permette di definire classi, proprietà e assiomi in modo dichiarativo, con l'importante differenza rispetto ai database di adottare l'**open world assumption**: un fatto non presente non è falso, è semplicemente sconosciuto).

7.2 Perché serve un formalismo diverso per pianificare

Is-a e Part-of sono relazioni essenzialmente **statiche**: descrivono cosa è qualcosa o di cosa è *fatto*, ma non catturano il concetto di **azione** né il cambiamento nel tempo. **Pianificare** significa invece costruire una sequenza di azioni che, partendo da uno stato iniziale, soddisfa un obiettivo — e per farlo servono strumenti che parlino esplicitamente di azioni, precondizioni ed effetti.

Il **Situation Calculus** fornisce il fondamento logico (in FOL) di questa idea. I concetti chiave:

- un'azione non è un predicato ma una **funzione** che restituisce un oggetto (es. $\text{Move}(R, L_1, L_2)$);
- una **situazione** è lo stato risultante dall'esecuzione di una sequenza di azioni a partire da una situazione iniziale S_0 ;
- un **fluente** è una proprietà che può cambiare valore con l'esecuzione delle azioni, ed ha sempre la situazione come parametro (es. $\text{Holds}(\text{At}(R, \text{Loc}), s)$).

La funzione $\text{Do}(\text{Azioni}, S)$ restituisce la situazione raggiunta applicando quella sequenza di azioni a partire da S , e permette di fare **proiezione**: ragionare in anticipo sugli effetti futuri di un piano, prima di eseguirlo davvero.

Ogni azione è descritta da due assiomi: un **assioma di applicabilità** (è eseguibile se e solo se le sue precondizioni sono vere) e un **assioma di effetto** (se eseguita, rende vere certe proprietà nella situazione risultante).

7.3 Il frame problem

Ecco un problema che sembra banale ma non lo è affatto: come rappresentiamo ciò che un'azione **non** modifica? Se sposto un blocco A da B a D, il sistema logico non ha alcun modo automatico di dedurre che tutto il resto del mondo (per esempio, che il blocco C è ancora sul tavolo) è rimasto invariato — dagli assiomi di applicabilità ed effetto si può ragionare solo su cosa *cambia*, non su cosa resta uguale.

Una prima soluzione, corretta ma poco pratica, è enumerare esplicitamente un “assioma di frame” per ogni combinazione azione×proprietà non toccata — al crescere del numero di azioni e proprietà, il numero di assiomi esplode. La soluzione elegante è un **assioma di stato successore** unico per ciascun fluente, che dice in un colpo solo: “un fluente è vero nella situazione risultante se e solo se l'azione lo rende vero, oppure era già vero e l'azione non lo ha reso falso”. Questo schema compatto evita del tutto l'enumerazione esplicita.

7.4 L'anomalia di Sussman: quando i sottoobiettivi litigano

Un approccio naturale per affrontare un obiettivo complesso è scomporlo in sottoobiettivi e risolverli uno alla volta. Ma questo non funziona sempre, e l'esempio canonico che lo dimostra è l'**anomalia di Sussman**: nel mondo dei blocchi, partiamo con A sopra B (impilati) e C isolato sul tavolo, e vogliamo raggiungere la pila A-B-C. Se proviamo a risolvere prima “B su C”, dobbiamo togliere A da B per liberarlo — disfacendo un pezzo di struttura che in realtà volevamo mantenere. Se proviamo prima “A su B” (già vero!), poi per mettere B su C dovremmo comunque smontare tutto e ricostruire.

La soluzione richiede **interleaving**: intrecciare i passi provenienti dai due sottopiani invece di eseguirli a blocchi separati (per esempio: sposto C, tolgo A da B, metto B su C, rimetto A su B). Questo dimostra un punto concettuale importante: i sottoobiettivi possono **interagire**, e un planner realmente efficace deve saperlo gestire, non limitarsi a concatenare soluzioni indipendenti.

Perché conta: L'anomalia di Sussman e il frame problem sono, in fondo, due facce della stessa medaglia: entrambi nascono dal fatto che rappresentare un mondo che cambia nel tempo è molto più difficile che rappresentare conoscenza statica (le tassonomie della prima parte del modulo). È esattamente per questa difficoltà, non per mancanza di eleganza teorica, che il Situation Calculus — pur fondamentale per capire i concetti — viene abbandonato nei planner reali a favore di rappresentazioni più specializzate come PDDL.

8 Machine Learning: classificazione, alberi di decisione, reti neurali

8.1 Le classi non esistono in natura

Prima di entrare negli algoritmi, vale la pena fissare un'idea filosofica importante che le slide sottolineano fin da subito: **le classi non sono un dato di natura**. Non sono raggruppamenti oggettivi e inequivocabili presenti negli esempi — sono **definite a tavolino** da chi costruisce il sistema, in base allo scopo. Lo stesso insieme di fiori si può raggruppare per colore, per forma, per specie: dipende da cosa interessa a chi progetta il classificatore. Conseguenza pratica importante: **il sistema non ha altra conoscenza oltre ai dati**, e impara esattamente ciò che c'è nei dati, così come sono etichettati — non ciò che il progettista *pensa* di aver messo nei dati. Se addestro un riconoscitore di “frutta” mostrandogli solo foto di mele, il sistema imparerà a riconoscere mele, non la frutta in generale.

Lo schema generale dell'**apprendimento supervisionato** è: un **training set** di esempi già etichettati viene usato per **indurre** un modello (una funzione che associa una descrizione a una classe); il modello viene poi valutato su un **test set** (dati mai visti in fase di apprendimento), confrontando classe predetta e classe reale.

La qualità si misura con la **matrice di confusione**: $\text{accuratezza} = (\text{predizioni corrette})/(\text{totale})$, $\text{error rate} = (\text{predizioni sbagliate})/(\text{totale})$. Ma occhio a un tranello classico: se il dataset è fortemente sbilanciato (9990 istanze di classe A, 10 di classe B), un classificatore banale che risponde *sempre* “A” ottiene un'accuratezza del 99.9% senza aver imparato assolutamente nulla di utile — un'accuratezza alta, da sola, **non** è garanzia di un buon modello: va sempre letta insieme alla composizione del dataset.

8.2 Alberi di decisione

Un albero di decisione classifica seguendo un percorso di test sugli attributi: i **nodi interni** sono test, le **foglie** sono decisioni (classi). Si costruiscono ricorsivamente con l'**algoritmo di Hunt**: se tutte le istanze in un nodo appartengono alla stessa classe, il nodo diventa foglia; altrimenti si sceglie un attributo, si crea un figlio per ogni suo valore possibile, e si ricorre su ciascun figlio. È una strategia **greedy** (nessun backtracking): ad ogni passo si sceglie localmente il “migliore” split, sperando che questo porti a un buon albero complessivo.

Ma cosa vuol dire “migliore” split? L'idea intuitiva è preferire gli split che producono figli più **puri** (concentrati su una sola classe). Per formalizzarla si usa l'**entropia**:

$$\text{Entropia}(t) = - \sum_i p(i|t) \log_2 p(i|t),$$

che vale 0 quando il nodo è perfettamente puro e raggiunge il suo massimo quando le classi sono equidistribuite. Il **guadagno** di uno split è la differenza fra l'impurità del nodo padre e la media pesata delle impurità dei figli — con l'entropia si chiama **information gain**, e si sceglie sempre lo split che lo massimizza.

C'è però una trappola importante da saper spiegare a voce: l'information gain **favorisce sistematicamente gli attributi con molti valori distinti**. Un identificatore univoco come il numero di matricola produce un figlio per ogni singola istanza, ciascuno perfettamente puro (una sola istanza dentro): l'information gain risulterebbe massimo, ma il modello ottenuto è pura memorizzazione (overfitting totale), incapace di generalizzare su istanze mai viste. Questo è un ottimo esempio da avere pronto se all'esame chiedono un limite delle misure di impurità classiche.

8.3 Il perceptron: un iperpiano che impara

Il modello più semplice di rete neurale, il **perceptron**, calcola una combinazione lineare pesata degli input e le applica una funzione di attivazione:

$$\text{net} = \sum_i w_i X_i, \quad Y = f(\text{net}).$$

Geometricamente, un perceptron codifica un **iperpiano** nello spazio degli input: tutto ciò che cade da un lato attiva il neurone (classe 1), tutto ciò che cade dall'altro no (classe 0). “Imparare”, per un perceptron, vuol dire proprio trovare la posizione corretta di questo iperpiano — regolando i pesi in base all'errore osservato, con la regola

$$w_j^{(k+1)} = w_j^{(k)} + \eta (d - o) x_j,$$

dove d è l'output desiderato, o quello ottenuto, ed η un fattore di apprendimento che impedisce all'algoritmo di “inseguire” solo l'ultimo esempio visto. Il teorema di Novikoff garantisce che questo processo converge se (e solo se) il problema è **linearmente separabile**.

Ed è proprio qui che sta il limite più famoso del perceptron: lo **XOR**. Le quattro combinazioni di input e output non sono separabili da nessuna singola retta nel piano — un solo perceptron non può mai imparare lo XOR. L'intuizione della soluzione è semplice da spiegare a parole: se un iperpiano non basta, bastano *due* iperpiani combinati insieme, e con abbastanza iperpiani si possono ritagliare regioni arbitrariamente complesse. Questa intuizione porta direttamente all'idea delle reti **multi-strato**.

8.4 MLP e backpropagation

Un **Multi-Layer Perceptron** organizza i neuroni in strati (input, uno o più *hidden*, output), collegati in modo *feed-forward* (il segnale scorre in una sola direzione). Con almeno tre livelli e neuroni sigmoide a sufficienza, un MLP diventa un **approssimatore universale di funzioni** — un fatto teorico notevole, che vale la pena saper enunciare.

L'apprendimento avviene tramite **backpropagation**, in due passate per ogni esempio: una **forward** (calcola l'output) e una **backward** (propaga l'errore all'indietro, dai neuroni di output verso quelli più vicini all'input, aggiornando i pesi). Il problema centrale che la backpropagation risolve è quello del **credit assignment**: sappiamo calcolare l'errore esatto solo per i neuroni di output (per cui conosciamo il target desiderato) — ma come distribuiamo la “colpa” dell'errore ai pesi degli strati precedenti, per cui non abbiamo un target diretto? La risposta è la **discesa del gradiente**: si muove ogni peso nella direzione che riduce l'errore globale,

$$E = \frac{1}{2} \sum_i (t_i - y_i)^2, \quad \Delta w_{ji} = -\eta \frac{\partial E}{\partial w_{ji}},$$

e per gli strati hidden l'errore si ottiene propagando all'indietro (da cui il nome dell'algoritmo) i contributi δ dei neuroni dello strato successivo, pesati sulla forza delle rispettive connessioni.

Perché conta: Vale la pena ricordare che tutto questo impianto — dagli alberi di decisione alle reti neurali — condivide lo stesso schema concettuale del modulo sugli agenti che apprendono (Modulo 1): un segnale di errore/feedback che guida la modifica del comportamento nel tempo. Cambia la tecnica specifica, ma l'idea di fondo — imparare dall'esperienza per migliorare — è la stessa che avete incontrato fin dall'inizio del corso.